```
Start coding or generate with AI.
```

Task

Build and evaluate a spam classification model using the dataset from "/content/sms+spam+collection.zip". The process should involve data loading, TF-IDF vectorization, stratified K-fold cross-validation with various models (Multinomial Naive Bayes, Logistic Regression, Linear SVM, AdaBoost, Voting, Stacking), selection of the best model, and evaluation on a holdout set. Save cross-validation results to "/content/ensemble_comparison.csv", holdout predictions to "/content/final_model_predictions.csv", and the final best model to "/content/best_spam_classifier.joblib".

Setup and Argument Parsing

Subtask:

Initialize the script structure, parse command-line arguments, and import all necessary libraries for data processing, model building, and evaluation.

Reasoning: The subtask requires importing all necessary libraries and setting up argument parsing. This step will import the specified libraries, define an ArgumentParser, add the required command-line arguments, parse them, and then print the parsed arguments for verification.

```
import os
import zipfile
import pandas as pd
import numpy as np
import argparse

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import VotingClassifier, StackingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.metrics import classification_report, confusion_matrix, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.pipeline import Pipeline
from joblib import dump, load

# Define ArgumentParser
parser = argparse.ArgumentParser(description='SMS Spam Classifier Script')

# Add command-line arguments
parser.add_argument('--data_path', type=str, default='/content/sms+spam+collection.zip', help='Path to the dataset zip file
parser.add_argument('--target_column', type=str, default='label', help='Name of the target column')
parser.add_argument('--k_fold', type=int, default=5, help='Number of folds for stratified K-fold cross-validation')
parser.add_argument('--holdout_ratio', type=float, default=0.2, help='Ratio for the holdout set split')
parser.add_argument('--random_state', type=int, default=42, help='Random state for reproducibility')

# Parse the arguments
args = parser.parse_args([]) # Passing an empty list to avoid parsing sys.argv in a notebook environment

# Print parsed arguments
print("Parsed arguments:")
for arg, value in vars(args).items():
    print(f"{arg}: {value}")
```

```
Parsed arguments:
data_path: /content/sms+spam+collection.zip
target_column: label
k_fold: 5
holdout_ratio: 0.2
random_state: 42
```

Data Loading and Preprocessing

Subtask:

Unzip the dataset, load it into a pandas DataFrame, and initialize a TF-IDF vectorizer.

Reasoning: The subtask requires unzipping the dataset, loading it into a pandas DataFrame with specified column names, and initializing a TF-IDF vectorizer. This code block will perform all these actions sequentially.

```
data_dir = '/content/data'
os.makedirs(data_dir, exist_ok=True)

# Unzip the dataset
with zipfile.ZipFile(args.data_path, 'r') as zip_ref:
    zip_ref.extract('SMSSpamCollection', data_dir)
    print(f"Extracted SMSSpamCollection to {data_dir}/")

# Load the extracted file into a pandas DataFrame
df_sms = pd.read_csv(os.path.join(data_dir, 'SMSSpamCollection'), sep='\t', names=['label', 'message'])
print("SMS dataset loaded successfully.")

# Display the first few rows of the DataFrame
print("First 5 rows of the dataset:")
print(df_sms.head())

# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(min_df=2, ngram_range=(1, 2), lowercase=True)
print("TF-IDF Vectorizer initialized.")
```

```
Extracted SMSSpamCollection to /content/data/
SMS dataset loaded successfully.
First 5 rows of the dataset:
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
TF-IDF Vectorizer initialized.
```

Data Splitting

Subtask:

Perform a stratified holdout split of the dataset into training and holdout sets.

Reasoning: The subtask requires separating features and target from the DataFrame, then performing a stratified train-test split using the specified `holdout_ratio` and `random_state` from the parsed arguments to create training and holdout sets.

```
X = df_sms['message']
y = df_sms['label']

# Perform stratified train-test split
X_train, X_holdout, y_train, y_holdout = train_test_split(X, y,
    test_size=args.holdout_ratio,
    stratify=y,
    random_state=args.random_state
)

print(f"Original dataset shape: {df_sms.shape}")
print(f"Training set shape: {X_train.shape}, {y_train.shape}")
print(f"Holdout set shape: {X_holdout.shape}, {y_holdout.shape}")
print("Data split into training and holdout sets successfully.")
```

```
Original dataset shape: (5572, 2)
Training set shape: (4457,), (4457,)
Holdout set shape: (1115,), (1115,)
Data split into training and holdout sets successfully.
```

Define Models and Pipelines

Subtask:

Create Pipeline objects for various models, incorporating the TF-IDF vectorizer, and define Voting and Stacking classifiers.

Reasoning: The subtask requires defining Pipeline objects for various models, including a TF-IDF vectorizer, and then creating Voting and Stacking classifiers. This code block will initialize these pipelines and ensemble models as specified.

```
print("Defining model pipelines...")
```

```
# 1. Pipeline for Multinomial Naive Bayes
mnb_pipeline = Pipeline([
    ('tfidf', tfidf_vectorizer),
    ('mnb', MultinomialNB())
])

# 2. Pipeline for Logistic Regression
lr_pipeline = Pipeline([
    ('tfidf', tfidf_vectorizer),
    ('lr', LogisticRegression(max_iter=1000, random_state=args.random_state))
])

# 3. Pipeline for LinearSVC wrapped in CalibratedClassifierCV
# LinearSVC needs dual=False for sparse data when n_samples > n_features
lsvc_pipeline = Pipeline([
    ('tfidf', tfidf_vectorizer),
    ('lsvc', CalibratedClassifierCV(LinearSVC(random_state=args.random_state, dual=False), method='isotonic'))
])

# 4. Pipeline for AdaBoostClassifier
ada_pipeline = Pipeline([
    ('tfidf', tfidf_vectorizer),
    ('ada', AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1, random_state=args.random_state), n_estimators=
])

# 5. List of base estimators for ensemble models
estimators = [
    ('mnb', mnb_pipeline),
    ('lr', lr_pipeline),
    ('lsvc', lsvc_pipeline), # Using the CalibratedClassifierCV wrapped LinearSVC
    ('ada', ada_pipeline)
]

# 6. Soft VotingClassifier
voter_soft = VotingClassifier(estimators=estimators, voting='soft', weights=[0.25, 0.25, 0.25, 0.25], verbose=False)

# 7. Hard VotingClassifier
voter_hard = VotingClassifier(estimators=estimators, voting='hard', verbose=False)

# 8. StackingClassifier
stack_clf = StackingClassifier(estimators=estimators,
                               final_estimator=LogisticRegression(max_iter=1000, random_state=args.random_state),
                               cv='prefit',
                               verbose=False)

print("Model pipelines and ensemble classifiers defined successfully.")
```

```
Defining model pipelines...
Model pipelines and ensemble classifiers defined successfully.
```

Cross-Validation Evaluation

Subtask:

Evaluate each defined model on the training set using stratified K-fold cross-validation and compute relevant metrics.

Reasoning: The subtask requires evaluating each defined model using stratified K-fold cross-validation. This code block will define the list of models, set up the cross-validation parameters, and iterate through each model to perform cross-validation and store the results.

```
print("Starting cross-validation evaluation for all models...")

# List of models to evaluate
models = [
    ('Multinomial Naive Bayes', mnb_pipeline),
    ('Logistic Regression', lr_pipeline),
    ('Linear SVC (Calibrated)', lsvc_pipeline),
    ('AdaBoost', ada_pipeline),
    ('Soft Voting Classifier', voter_soft),
    ('Hard Voting Classifier', voter_hard),
    ('Stacking Classifier', stack_clf)
]

# Initialize Stratified K-Fold
skf = StratifiedKFold(n_splits=args.k_fold, shuffle=True, random_state=args.random_state)

# Define scoring metrics
scoring = ['precision_weighted', 'recall_weighted', 'f1_weighted', 'roc_auc_weighted']
```

```
# List to store results
cv_results = []

# Perform cross-validation for each model
for name, model in models:
    print(f"\nEvaluating {name}...")
    results = cross_validate(model, X_train, y_train, cv=skf, scoring=scoring, n_jobs=-1, verbose=1)

    # Store mean and std of each metric
    cv_results.append({
        'Model': name,
        'Precision_mean': np.mean(results['test_precision_weighted']),
        'Precision_std': np.std(results['test_precision_weighted']),
        'Recall_mean': np.mean(results['test_recall_weighted']),
        'Recall_std': np.std(results['test_recall_weighted']),
        'F1_mean': np.mean(results['test_f1_weighted']),
        'F1_std': np.std(results['test_f1_weighted']),
        'ROC_AUC_mean': np.mean(results['test_roc_auc_weighted']),
        'ROC_AUC_std': np.std(results['test_roc_auc_weighted'])
    })
    print(f"{name} evaluation complete.")

print("All models evaluated successfully.")
```

```
Starting cross-validation evaluation for all models...

Evaluating Multinomial Naive Bayes...
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_scorer.py in get_scorer(scoring)
    439             else:
--> 440                 scorer = copy.deepcopy(_SCORERS[scoring])
    441         except KeyError:

KeyError: 'roc_auc_weighted'

During handling of the above exception, another exception occurred:

ValueError                                Traceback (most recent call last)
                              9 frames
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_scorer.py in get_scorer(scoring)
    440                 scorer = copy.deepcopy(_SCORERS[scoring])
    441         except KeyError:
--> 442             raise ValueError(
    443                 "%r is not a valid scoring value. "
    444                 "Use sklearn.metrics.get_scorer_names() "

ValueError: 'roc_auc_weighted' is not a valid scoring value. Use sklearn.metrics.get_scorer_names() to get valid options.
```

Reasoning: The previous code failed because 'roc_auc_weighted' is not a valid scoring value in `sklearn.metrics.cross_validate`. I need to change it to 'roc_auc' for binary classification to fix this error.

```
print("Starting cross-validation evaluation for all models...")

# List of models to evaluate
models = [
    ('Multinomial Naive Bayes', mnb_pipeline),
    ('Logistic Regression', lr_pipeline),
    ('Linear SVC (Calibrated)', lsvc_pipeline),
    ('AdaBoost', ada_pipeline),
    ('Soft Voting Classifier', voter_soft),
    ('Hard Voting Classifier', voter_hard),
    ('Stacking Classifier', stack_clf)
]

# Initialize Stratified K-Fold
skf = StratifiedKFold(n_splits=args.k_fold, shuffle=True, random_state=args.random_state)

# Define scoring metrics
scoring = ['precision_weighted', 'recall_weighted', 'f1_weighted', 'roc_auc']

# List to store results
cv_results = []

# Perform cross-validation for each model
for name, model in models:
    print(f"\nEvaluating {name}...")
    results = cross_validate(model, X_train, y_train, cv=skf, scoring=scoring, n_jobs=-1, verbose=1)

    # Store mean and std of each metric
```

```
    cv_results.append({
        'Model': name,
        'Precision_mean': np.mean(results['test_precision_weighted']),
        'Precision_std': np.std(results['test_precision_weighted']),
        'Recall_mean': np.mean(results['test_recall_weighted']),
        'Recall_std': np.std(results['test_recall_weighted']),
        'F1_mean': np.mean(results['test_f1_weighted']),
        'F1_std': np.std(results['test_f1_weighted']),
        'ROC_AUC_mean': np.mean(results['test_roc_auc']),
        'ROC_AUC_std': np.std(results['test_roc_auc'])
    })
    print(f"{name} evaluation complete.")

print("All models evaluated successfully.")
```

```
Starting cross-validation evaluation for all models...

Evaluating Multinomial Naive Bayes...
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    8.2s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Multinomial Naive Bayes evaluation complete.

Evaluating Logistic Regression...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    1.5s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Logistic Regression evaluation complete.

Evaluating Linear SVC (Calibrated)...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    1.6s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Linear SVC (Calibrated) evaluation complete.

Evaluating AdaBoost...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    6.6s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
AdaBoost evaluation complete.

Evaluating Soft Voting Classifier...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:   11.6s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Soft Voting Classifier evaluation complete.

Evaluating Hard Voting Classifier...
---------------------------------------------------------------------------
_RemoteTraceback                          Traceback (most recent call last)
_RemoteTraceback:
"""
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/joblib/externals/loky/process_executor.py", line 490, in _process_worker
    r = call_item()
        ^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/joblib/externals/loky/process_executor.py", line 291, in __call__
    return self.fn(*self.args, **self.kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/joblib/parallel.py", line 607, in __call__
    return [func(*args, **kwargs) for func, args, kwargs in self.items]
            ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/parallel.py", line 139, in __call__
    return self.function(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 888, in _fit_and_score
    test_scores = _score(
                  ^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 949, in _score
    scores = scorer(estimator, X_test, y_test, **score_params)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_scorer.py", line 126, in __call__
    cache = {} if self._use_cache(estimator) else None
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_scorer.py", line 166, in _use_cache
    _check_response_method(estimator, scorer._response_method).__name__
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py", line 2283, in _check_response_method
    raise AttributeError(
AttributeError: VotingClassifier has none of the following attributes: decision_function, predict_proba.
"""

The above exception was the direct cause of the following exception:

AttributeError                            Traceback (most recent call last)
/tmp/ipython-input-782/4266761185.py in <cell line: 0>()
     24 for name, model in models:
     25     print(f"\nEvaluating {name}...")
---> 26     results = cross_validate(model, X_train, y_train, cv=skf, scoring=scoring, n_jobs=-1, verbose=1)
     27
     28     # Store mean and std of each metric

                              8 frames
/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in _return_or_raise(self)
    771         try:
    772             if self.status == TASK_ERROR:
--> 773                 raise self._result
    774             return self._result
    775         finally:

AttributeError: VotingClassifier has none of the following attributes: decision_function, predict_proba.
```

Reasoning: The previous code failed because the `Hard Voting Classifier` does not provide probability estimates (i.e., it lacks a `predict_proba` method), which is required by the `roc_auc` scoring metric. To fix this while evaluating all models, the `roc_auc` metric will be conditionally excluded for the `Hard Voting Classifier`.

```
print("Starting cross-validation evaluation for all models...")

# List of models to evaluate
models = [
    ('Multinomial Naive Bayes', mnb_pipeline),
    ('Logistic Regression', lr_pipeline),
    ('Linear SVC (Calibrated)', lsvc_pipeline),
    ('AdaBoost', ada_pipeline),
    ('Soft Voting Classifier', voter_soft),
    ('Hard Voting Classifier', voter_hard),
    ('Stacking Classifier', stack_clf)
]

# Initialize Stratified K-Fold
skf = StratifiedKFold(n_splits=args.k_fold, shuffle=True, random_state=args.random_state)

# Define common scoring metrics
common_scoring_metrics = ['precision_weighted', 'recall_weighted', 'f1_weighted']
# Define scoring metrics for models that support probability prediction
probabilistic_scoring_metrics = common_scoring_metrics + ['roc_auc']

# List to store results
cv_results = []

# Perform cross-validation for each model
for name, model in models:
    print(f"\nEvaluating {name}...")

    # Determine scoring metrics based on model type
    current_scoring = probabilistic_scoring_metrics
    if name == 'Hard Voting Classifier':
        # Hard Voting Classifier does not have predict_proba, so ROC AUC cannot be calculated directly
        current_scoring = common_scoring_metrics
        print(f"Note: ROC AUC will be skipped for '{name}' as it does not provide probability estimates for `cross_validate

    try:
        results = cross_validate(model, X_train, y_train, cv=skf, scoring=current_scoring, n_jobs=-1, verbose=1)

        model_result = {
            'Model': name,
            'Precision_mean': np.mean(results['test_precision_weighted']),
            'Precision_std': np.std(results['test_precision_weighted']),
            'Recall_mean': np.mean(results['test_recall_weighted']),
            'Recall_std': np.std(results['test_recall_weighted']),
            'F1_mean': np.mean(results['test_f1_weighted']),
            'F1_std': np.std(results['test_f1_weighted']),
        }

        if 'roc_auc' in current_scoring:
            model_result['ROC_AUC_mean'] = np.mean(results['test_roc_auc'])
            model_result['ROC_AUC_std'] = np.std(results['test_roc_auc'])
        else:
            model_result['ROC_AUC_mean'] = np.nan
            model_result['ROC_AUC_std'] = np.nan

        cv_results.append(model_result)
        print(f"{name} evaluation complete.")

    except AttributeError as e:
        print(f"Error evaluating {name}: {e}")
        print(f"Skipping {name} due to an AttributeError during cross-validation.")
        # Store NaN for metrics if an error occurs
        cv_results.append({
            'Model': name,
            'Precision_mean': np.nan,
            'Precision_std': np.nan,
            'Recall_mean': np.nan,
            'Recall_std': np.nan,
            'F1_mean': np.nan,
            'F1_std': np.nan,
            'ROC_AUC_mean': np.nan,
            'ROC_AUC_std': np.nan
        })
    except ValueError as e:
        print(f"Error evaluating {name}: {e}")
        print(f"Skipping {name} due to a ValueError during cross-validation.")
```

```
        cv_results.append({
            'Model': name,
            'Precision_mean': np.nan,
            'Precision_std': np.nan,
            'Recall_mean': np.nan,
            'Recall_std': np.nan,
            'F1_mean': np.nan,
            'F1_std': np.nan,
            'ROC_AUC_mean': np.nan,
            'ROC_AUC_std': np.nan
        })

print("All models evaluated successfully.")
```

```
Evaluating Logistic Regression...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    1.4s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Logistic Regression evaluation complete.

Evaluating Linear SVC (Calibrated)...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    1.8s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Linear SVC (Calibrated) evaluation complete.

Evaluating AdaBoost...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    8.2s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
AdaBoost evaluation complete.

Evaluating Soft Voting Classifier...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:   10.9s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Soft Voting Classifier evaluation complete.

Evaluating Hard Voting Classifier...
Note: ROC AUC will be skipped for 'Hard Voting Classifier' as it does not provide probability estimates for `cross_validat
Hard Voting Classifier evaluation complete.

Evaluating Stacking Classifier...
Error evaluating Stacking Classifier:
All the 5 fits failed.
It is very likely that your model is misconfigured.
You can try to debug the error by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py", line 63, in inner_f
    return f(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_stacking.py", line 717, in fit
    return super().fit(X, y_encoded, **fit_params)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_stacking.py", line 206, in fit
    check_is_fitted(estimator)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py", line 1757, in check_is_fitted
    raise NotFittedError(msg % {"name": type(estimator).__name__})
sklearn.exceptions.NotFittedError: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before

Skipping Stacking Classifier due to a ValueError during cross-validation.
All models evaluated successfully.
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    9.0s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    0.1s finished
```

Reasoning: The previous code failed because the `StackingClassifier` was initialized with `cv='prefit'`, but its base estimators were not pre-fitted for `cross_validate`. To fix this, the `StackingClassifier`'s `cv` parameter needs to be set to a valid cross-validation strategy, such as `skf` (StratifiedKFold) or an integer, so it can correctly fit its base estimators during each fold of the cross-validation process.

```
print("Starting cross-validation evaluation for all models...")

# List of models to evaluate
models = [
    ('Multinomial Naive Bayes', mnb_pipeline),
    ('Logistic Regression', lr_pipeline),
    ('Linear SVC (Calibrated)', lsvc_pipeline),
    ('AdaBoost', ada_pipeline),
```

```
    ('AdaBoost', ada_pipeline),
    ('Soft Voting Classifier', voter_soft),
    ('Hard Voting Classifier', voter_hard)
]

# Initialize Stratified K-Fold
skf = StratifiedKFold(n_splits=args.k_fold, shuffle=True, random_state=args.random_state)

# Redefine StackingClassifier with a proper 'cv' strategy for cross_validate
# The base estimators (mnb_pipeline, lr_pipeline, lsvc_pipeline, ada_pipeline) are themselves pipelines
# The 'cv' parameter in StackingClassifier should handle the internal cross-validation for meta-feature generation
# Changing 'cv="prefit"' to 'cv=skf' (or an integer) allows StackingClassifier to correctly fit its estimators within cross_
stack_clf = StackingClassifier(estimators=[
    ('mnb', mnb_pipeline.steps[-1][1]), # Pass only the classifier part for stacking
    ('lr', lr_pipeline.steps[-1][1]),
    ('lsvc', lsvc_pipeline.steps[-1][1]),
    ('ada', ada_pipeline.steps[-1][1])
],
final_estimator=LogisticRegression(max_iter=1000, random_state=args.random_state),
cv=skf, # Use the defined StratifiedKFold for internal cross-validation
verbose=False)

# Add the corrected Stacking Classifier to the models list
models.append(('Stacking Classifier', Pipeline([('tfidf', tfidf_vectorizer), ('stack', stack_clf)])))


# Define common scoring metrics
common_scoring_metrics = ['precision_weighted', 'recall_weighted', 'f1_weighted']
# Define scoring metrics for models that support probability prediction
probabilistic_scoring_metrics = common_scoring_metrics + ['roc_auc']

# List to store results
cv_results = []

# Perform cross-validation for each model
for name, model in models:
    print(f"\nEvaluating {name}...")

    # Determine scoring metrics based on model type
    current_scoring = probabilistic_scoring_metrics
    if name == 'Hard Voting Classifier':
        # Hard Voting Classifier does not have predict_proba, so ROC AUC cannot be calculated directly
        current_scoring = common_scoring_metrics
        print(f"Note: ROC AUC will be skipped for '{name}' as it does not provide probability estimates for `cross_validate`

    # StackingClassifier requires its base estimators to receive transformed data, so tfidf must be applied before stacking
    # The previous stacking definition had pipelines as estimators. When stacking, the base estimators should be classifiers
    # and the vectorization step should ideally be external or part of a full pipeline that includes stacking.
    # Given the existing structure, for StackingClassifier, it's better to separate TF-IDF from the base estimators
    # and apply it as a first step to the entire StackingClassifier pipeline.

    # For the StackingClassifier in this loop, we need to ensure the TF-IDF vectorizer is applied correctly.
    # The models list now contains a Pipeline wrapping StackingClassifier, which should handle TFIDF.

    try:
        results = cross_validate(model, X_train, y_train, cv=skf, scoring=current_scoring, n_jobs=-1, verbose=1)

        model_result = {
            'Model': name,
            'Precision_mean': np.mean(results['test_precision_weighted']),
            'Precision_std': np.std(results['test_precision_weighted']),
            'Recall_mean': np.mean(results['test_recall_weighted']),
            'Recall_std': np.std(results['test_recall_weighted']),
            'F1_mean': np.mean(results['test_f1_weighted']),
            'F1_std': np.std(results['test_f1_weighted']),
        }

        if 'roc_auc' in current_scoring:
            model_result['ROC_AUC_mean'] = np.mean(results['test_roc_auc'])
            model_result['ROC_AUC_std'] = np.std(results['test_roc_auc'])
        else:
            model_result['ROC_AUC_mean'] = np.nan
            model_result['ROC_AUC_std'] = np.nan

        cv_results.append(model_result)
        print(f"{name} evaluation complete.")

    except AttributeError as e:
        print(f"Error evaluating {name}: {e}")
        print(f"Skipping {name} due to an AttributeError during cross-validation.")
        # Store NaN for metrics if an error occurs
        cv_results.append({
            'Model': name,
```

```
            'Precision_mean': np.nan,
            'Precision_std': np.nan,
            'Recall_mean': np.nan,
            'Recall_std': np.nan,
            'F1_mean': np.nan,
            'F1_std': np.nan,
            'ROC_AUC_mean': np.nan,
            'ROC_AUC_std': np.nan
        })
    except ValueError as e:
        print(f"Error evaluating {name}: {e}")
        print(f"Skipping {name} due to a ValueError during cross-validation.")
        cv_results.append({
            'Model': name,
            'Precision_mean': np.nan,
            'Precision_std': np.nan,
            'Recall_mean': np.nan,
            'Recall_std': np.nan,
            'F1_mean': np.nan,
            'F1_std': np.nan,
            'ROC_AUC_mean': np.nan,
            'ROC_AUC_std': np.nan
        })

print("All models evaluated successfully.")
```

```
Starting cross-validation evaluation for all models...

Evaluating Multinomial Naive Bayes...
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    2.3s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Multinomial Naive Bayes evaluation complete.

Evaluating Logistic Regression...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    2.7s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Logistic Regression evaluation complete.

Evaluating Linear SVC (Calibrated)...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    1.7s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Linear SVC (Calibrated) evaluation complete.

Evaluating AdaBoost...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    8.2s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
AdaBoost evaluation complete.

Evaluating Soft Voting Classifier...
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:   10.5s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Soft Voting Classifier evaluation complete.

Evaluating Hard Voting Classifier...
Note: ROC AUC will be skipped for 'Hard Voting Classifier' as it does not provide probability estimates for `cross_validate`
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:    9.2s finished
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
Hard Voting Classifier evaluation complete.

Evaluating Stacking Classifier...
Stacking Classifier evaluation complete.
All models evaluated successfully.
[Parallel(n_jobs=-1)]: Done   5 out of   5 | elapsed:   45.8s finished
```

Reasoning: The previous code executed successfully. Now, I need to convert the `cv_results` list into a pandas DataFrame and save it to a CSV file as specified in the task description. This will store the cross-validation results for all models.

```
print("Saving cross-validation results to CSV...")

# Convert results to DataFrame
cv_results_df = pd.DataFrame(cv_results)

# Sort by F1_mean for better readability
cv_results_df = cv_results_df.sort_values(by='F1_mean', ascending=False)

# Save the DataFrame to a CSV file
cv_results_df.to_csv('/content/ensemble_comparison.csv', index=False)

print("Cross-validation results saved to /content/ensemble_comparison.csv")
print("Cross-validation results (sorted by F1_mean):")
print(cv_results_df)
```

```
Saving cross-validation results to CSV...
Cross-validation results saved to /content/ensemble_comparison.csv
Cross-validation results (sorted by F1_mean):
                     Model  Precision_mean  Precision_std  Recall_mean  \
2  Linear SVC (Calibrated)        0.987388       0.001941     0.987435
6      Stacking Classifier        0.986683       0.001665     0.986762
4   Soft Voting Classifier        0.981200       0.001865     0.980929
5   Hard Voting Classifier        0.967417       0.003996     0.966122
1      Logistic Regression        0.966056       0.002058     0.964775
0  Multinomial Naive Bayes        0.965534       0.001730     0.964102
3                 AdaBoost        0.943247       0.006443     0.943907

   Recall_std   F1_mean    F1_std  ROC_AUC_mean  ROC_AUC_std
2    0.001932  0.987381  0.001933      0.993314     0.002379
6    0.001650  0.986652  0.001659      0.991851     0.003881
4    0.001871  0.980346  0.001980      0.991519     0.004864
5    0.004330  0.963952  0.004950           NaN          NaN
1    0.002189  0.962488  0.002502      0.991629     0.002946
0    0.001873  0.961685  0.002139      0.982277     0.003620
3    0.005509  0.939255  0.006122      0.955561     0.008719
```

Best Model Selection and Retraining

Subtask:

Identify the best performing model based on cross-validation results and retrain it on the entire training dataset.

Reasoning: The subtask requires identifying the best performing model from the cross-validation results and retraining it on the entire training dataset. This code block will identify the model with the highest F1_mean, extract its pipeline from the `models` list, and then fit this pipeline on `X_train` and `y_train`.

```
print("Identifying the best model and retraining...")

# Identify the best model based on F1_mean
best_model_row = cv_results_df.loc[cv_results_df['F1_mean'].idxmax()]
best_model_name = best_model_row['Model']

print(f"Best model identified: {best_model_name}")

# Extract the best model's pipeline from the 'models' list
best_model_pipeline = None
for name, pipeline in models:
    if name == best_model_name:
        best_model_pipeline = pipeline
        break

if best_model_pipeline is None:
    raise ValueError(f"Could not find pipeline for best model: {best_model_name}")

# Retrain the best model on the entire training dataset
print(f"Retraining {best_model_name} on the entire training dataset...")
best_model_fitted = best_model_pipeline.fit(X_train, y_train)

print(f"'{best_model_name}' has been successfully retrained.")
```

```
Identifying the best model and retraining...
Best model identified: Linear SVC (Calibrated)
Retraining Linear SVC (Calibrated) on the entire training dataset...
'Linear SVC (Calibrated)' has been successfully retrained.
```

Holdout Set Evaluation and Prediction

Subtask:

Use the retrained best model to make predictions on the holdout set, generate evaluation metrics, and save predictions to a CSV file.

Reasoning: First, I need to predict the labels and probabilities (if supported) for the holdout set using the `best_model_fitted`. Then, I will print the classification report and confusion matrix to evaluate the model's performance on the holdout set.

```
print("Evaluating the best model on the holdout set...")

# Predict labels on the holdout set
y_pred = best_model_fitted.predict(X_holdout)
print("Predictions made on holdout set.")
```

```
# Predict probabilities for the 'spam' class if supported
spam_probability = np.nan
if hasattr(best_model_fitted, 'predict_proba'):
    y_pred_proba = best_model_fitted.predict_proba(X_holdout)
    # Get the index of the 'spam' class
    spam_class_idx = np.where(best_model_fitted.classes_ == 'spam')[0][0]
    spam_probability = y_pred_proba[:, spam_class_idx]
    print("Probabilities predicted for 'spam' class.")
else:
    print("Model does not support predict_proba, spam probabilities will be NaN.")

# Print classification report
print("\nClassification Report on Holdout Set:")
print(classification_report(y_holdout, y_pred))

# Print confusion matrix
print("\nConfusion Matrix on Holdout Set:")
print(confusion_matrix(y_holdout, y_pred))
```

```
Evaluating the best model on the holdout set...
Predictions made on holdout set.
Probabilities predicted for 'spam' class.

Classification Report on Holdout Set:
              precision    recall  f1-score   support

         ham       0.99      1.00      0.99       966
        spam       0.98      0.93      0.96       149

    accuracy                           0.99      1115
   macro avg       0.98      0.96      0.97      1115
weighted avg       0.99      0.99      0.99      1115


Confusion Matrix on Holdout Set:
[[963   3]
 [ 10 139]]
```

Reasoning: The next step is to create a pandas DataFrame with the holdout messages' indices, actual labels, predicted labels, and predicted spam probabilities, and then save this DataFrame to a CSV file as specified in the instructions.

```
print("Creating and saving holdout predictions DataFrame...")

# Create a DataFrame for predictions
holdout_predictions_df = pd.DataFrame({
    'MessageId': X_holdout.index, # Assuming X_holdout retains original DataFrame index
    'Actual': y_holdout,
    'Predicted': y_pred,
    'Probability': spam_probability
})

# Reset index to make 'MessageId' a regular column if X_holdout.index is not unique or appropriate
holdout_predictions_df = holdout_predictions_df.reset_index(drop=True)

# Save the DataFrame to a CSV file
holdout_predictions_df.to_csv('/content/final_model_predictions.csv', index=False)

print("Holdout predictions saved to /content/final_model_predictions.csv")
print("First 5 rows of final model predictions:")
print(holdout_predictions_df.head())
```

```
Creating and saving holdout predictions DataFrame...
Holdout predictions saved to /content/final_model_predictions.csv
First 5 rows of final model predictions:
   MessageId Actual Predicted  Probability
0       2825    ham       ham     0.000386
1       3695    ham       ham     0.000000
2       3904    ham       ham     0.000000
3        576   spam      spam     1.000000
4       2899    ham       ham     0.004877
```

Reasoning: The next step is to save the retrained best model to the specified joblib file, as required by the overall task.

```
print("Saving the final best model...")
dump(best_model_fitted, '/content/best_spam_classifier.joblib')
```